

Problem 13 : Give an algorithm for deleting an element (assuming data is given) from Binary tree.

Solution : The deletion of a node in Binary tree can be implemented as

- Starting at root, find the node which we want to delete.
- Find the deepest node in the tree
- Replace the deepest node's data with node to be deleted
- Then delete the deepest node.

Algorithm

A Binary Tree has no ordering, So we :

1. Find the node to be deleted
 2. Find the deepest (last) node
 3. Replace the data of the node to delete with deepest node's data
 4. Delete the deepest node
-
1. if $root == null$, return
 2. if tree has only one node
 - if $root.data == key$,
Set $root == null$ and return
 3. Initialize :
 - Queue Q
 - targetNode = null
 - DeepestNode = null
 - Parent of Deepest = null
 4. Perform level order traversal
 - For Each node
 - if $node's\ data == key$, store reference in targetNode

- Track last node visited as deepest Node
- Track parent of deepest Node

5. if target Node == Null, Element not found → return

6. Copy deepestNode.data into targetNode.data

7. Delete the deepest node :

- if deepest node is left child → Set parent.left = null
- Else → Set parent.right = null

Code

```
public static void deleteNode(TreeNode root, int key) {
    if (root == null)
        return;

    if (root.left == null && root.right == null) {
        if (root.data == key)
            root = null;
        return;
    }

    Queue q = new LinkedList<>();
    q.add(root);

    TreeNode target = null;
    TreeNode curr = null;
    TreeNode parent = null;

    while (!q.isEmpty()) {
        curr = q.poll();

        if (curr.data == key)
            target = curr;

        if (curr.left != null) {
            parent = curr;
            q.add(curr.left);
        }

        if (curr.right != null) {
```

```

        parent = curr;
        q.add(curr.right);
    }
}

if (target != null) {
    // replace target data with deepest node data
    target.data = curr.data;

    // delete deepest node
    if (parent.right == curr)
        parent.right = null;
    else
        parent.left = null;
}
}

```

Problem 14: Give an algorithm for finding the number of leaves in the Binary tree without using recursion.

Solution: The Set of nodes whose both left & right children are Null are called leaf nodes.

Algorithm

1. if root == null, return 0
2. Create an Empty Queue Q
3. Enqueue root into Q
4. Initialize leafCount = 0
5. While Q is not Empty :
 - Dequeue a node curr
 - if curr.left == null AND curr.right == null.
 - increment leafCount
 - if curr.left exists, Enqueue it.
 - if curr.right exists, Enqueue it.
6. Return leafCount.

code

```
public static int countLeaves(TreeNode root) {
    if (root == null)
        return 0;

    Queue<TreeNode> q = new LinkedList<>();
    q.add(root);

    int leafCount = 0;

    while (!q.isEmpty()) {
        TreeNode cur = q.poll();

        if (cur.left == null && cur.right == null)
            leafCount++;

        if (cur.left != null)
            q.add(cur.left);
        if (cur.right != null)
            q.add(cur.right);
    }
    return leafCount;
}
```

Problem 15 Give an algorithm for finding the number of full nodes in the Binary tree without using Recursion

Solution :- The set of all nodes with left and right children are called full nodes.

Algorithm : Count Full Nodes (root)

1. if root == null, return 0,
2. Create an Empty Queue Q
3. Enqueue root into Q

4. Initialize full count = 0
5. While Q is not Empty
 - Dequeue a node curr
 - if curr.left != Null AND curr.right != Null
 - increment full count
 - if curr.left exists, Enqueue it
 - if curr.right exists, Enqueue it
6. Return full count.

code

```
public static int countFullNodes(TreeNode root) {  
    if (root == null)  
        return 0;  
  
    Queue<TreeNode> q = new LinkedList<>();  
    q.add(root);  
  
    int fullCount = 0;  
  
    while (!q.isEmpty()) {  
        TreeNode cur = q.poll();  
  
        if (cur.left != null && cur.right != null)  
            fullCount++;  
  
        if (cur.left != null)  
            q.add(cur.left);  
        if (cur.right != null)  
            q.add(cur.right);  
    }  
    return fullCount;  
}
```